



Paralelización con OpenMP

Ecuación de Poisson en 2D

Sistemas Distribuidos — Análisis de Escalabilidad HPC

Camilo Huertas Sebastián Rodríguez

Programa Académico de Física

Universidad Distrital Francisco José de Caldas

Docente: Carlos Andrés Gómez Vasco | 24 de marzo de 2026

Contenido

- 1 Objetivo y Sistema HPC
- 2 Código Base Secuencial
- 3 Análisis de Escalabilidad
- 4 Actividad 1 — `parallel for`
- 5 Actividad 2 — `collapse(2)`
- 6 Actividad 3 — `sections`
- 7 Actividad 4 — `schedule`
- 8 Actividad 5 — Sincronización Explícita
- 9 Actividad 6 — `critical` vs `atomic`
- 10 Actividad 7 — `task`
- 11 Comparación Final
- 12 Validación Numérica
- 13 Conclusiones

Objetivo del Proyecto

Objetivo

Aplicar directivas de **OpenMP** para paralelizar la **Ecuación de Poisson 2D** (diferencias finitas). Se evaluará la *escalabilidad*, cuellos de botella de memoria y límites de aceleración (Ley de Amdahl) mediante un barrido paramétrico masivo.

Problema Físico

La ecuación de Poisson en 2D:

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = f(x, y)$$

- Término fuente $f(x, y) = 4$.
- Esquema centrado *in-place*.
- Tolerancia iterativa $TOL = 10^{-6}$.

Directivas Evaluadas

- `parallel for + reduction`
- `collapse(2)`
- `sections`
- `schedule(static)`
- `nowait, barrier, single, critical`
- `atomic` | `task`

Especificaciones del Sistema HPC

Hardware del Servidor

Componente	Especificación
Procesador	AMD Ryzen Threadripper 3990X
Arquitectura	x86_64
Núcleos / Hilos	64 Físicos / 128 Lógicos
Caché L3	256 MiB

Relevancia Arquitectónica

La caché L3 de 256 MiB es clave: al superarse, el ancho de banda a la RAM se vuelve el **cuello de botella dominante** (Von Neumann).

Metodología de Pruebas

- Mallas: 64^2 , 128^2 , 256^2 , 512^2 , **1024^2**
- Hilos:
 $p \in \{1, 4, 8, 16, 32, 64\}$

Sweet Spot Identificado

Para mallas ≤ 256 : óptimo en **32 hilos**. Para mallas ≥ 512 : el máximo rendimiento se alcanza con **64 hilos** ($43,9\times$ en 1024^2).

Código Base: Línea de Referencia

Discretización (Diferencias Finitas)

$$T_{i,j}^{n+1} = \frac{(T_{i+1,j} + T_{i-1,j})k^2 + (T_{i,j+1} + T_{i,j-1})h^2 - f_{i,j}h^2k^2}{2(h^2 + k^2)}$$

```

1 while (delta > TOL) {
2     delta = 0.0;
3     for (int i = 1; i < M; ++i)
4         for (int j = 1; j < N; ++j) {
5             double T_old = T[i][j];
6             T[i][j] = (((T[i+1][j]+T[i-1][j])*k*k)
7                 + ((T[i][j+1]+T[i][j-1])*h*h)
8                 - (source[i][j]*h*h*k*k))
9                 / (2.0*(h*h+k*k));
10            double diff = abs(T[i][j]-T_old);
11            if (diff > delta) delta = diff;
12        }
13        iterations++;
14    }

```

Listing 1: Bucle iterativo secuencial

Resultados (1 hilo)

Malla	Tiempo (s)	Iter.
64 ²	0.088	3 136
128 ²	1.194	10 244
256 ²	15.132	31 844
512 ²	176.55	91 868
1024 ²	1822.81	236 161

Observación

Las iteraciones crecen como $\mathcal{O}(N^2)$.
 Convergencia lenta típica de Gauss-Seidel.
 La malla 1024² requiere ≈ 30 min
 secuencial.

Curvas de Rendimiento — Mallas de Baja Escala

Rendimiento OpenMP - Malla 64x64

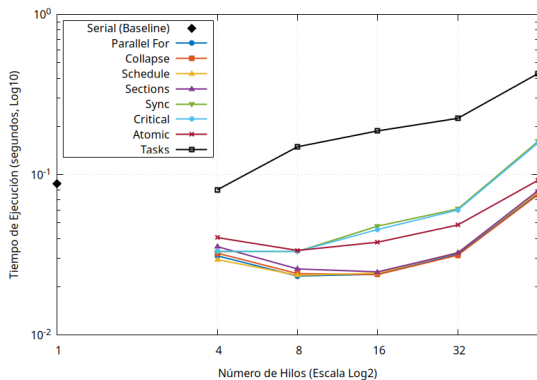


Figura: Malla 64²: Saturación temprana a 32 hilos.

Rendimiento OpenMP - Malla 128x128

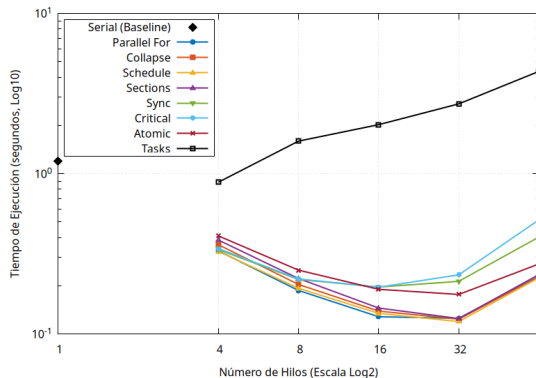


Figura: Malla 128²: Transición del régimen de escalabilidad.

Curvas de Rendimiento — Mallas de Media Escala

Rendimiento OpenMP - Malla 256x256

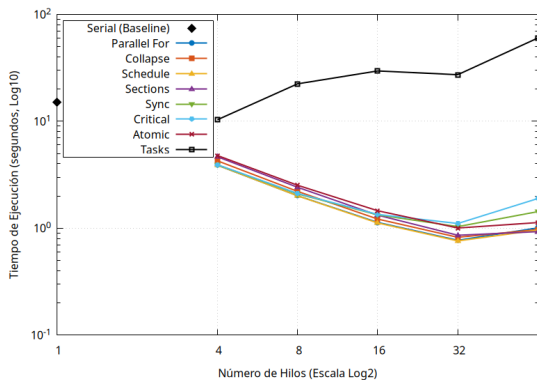


Figura: Malla 256²: Mejora en la fracción paralela α .

Rendimiento OpenMP - Malla 512x512

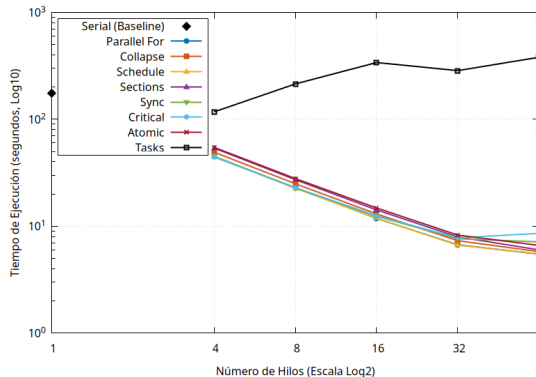
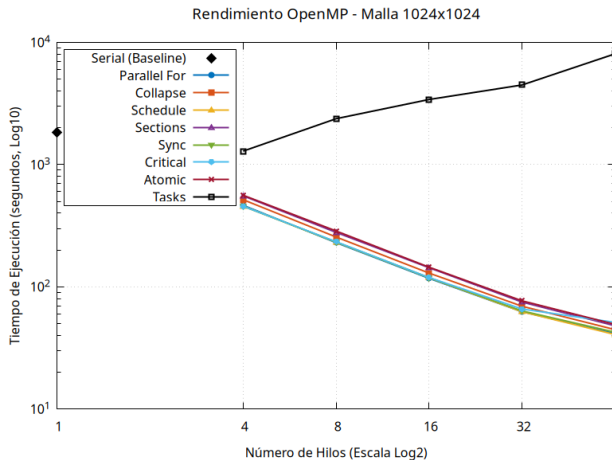


Figura: Malla 512²: Desplazamiento del óptimo a 64 hilos.

Análisis de Rendimiento Crítico (1024^2)



Hallazgos Clave

- Mallas ≤ 256 : óptimo en **32 hilos**.
- Mallas ≥ 512 : óptimo en **64 hilos**.
- `parallel for` logra **43,9×** ($E \approx 68\%$).
- `task` colapsa por excesivo *overhead*.

Ley de Amdahl

$$S(p) = \frac{1}{(1 - \alpha) + \frac{\alpha}{p}}$$

A medida que N crece, el costo computacional domina sobre la latencia de sincronización, incrementando α .

Figura: Resolución 1024^2 . Máximo *speedup* observado

Act. 1 — #pragma omp parallel for

```

1 #pragma omp parallel for reduction(max:delta)
2 for (int i = 1; i < M; ++i) {
3     for (int j = 1; j < N; ++j) {
4         double T_old = T[i][j];
5         T[i][j] = (((T[i+1][j]+T[i-1][j])*k*k) + ((T[i][j+1]+T[i][j-1])*h*h)
6                 -(source[i][j]*h*h*k*k)) / (2.0*(h*h+k*k));
7         double diff = abs(T[i][j]-T_old);
8         if (diff > delta) delta = diff;
9     }
10 }
11 iterations++;

```

Listing 2: Paralelización del bucle principal

¿Por qué reduction(max:delta)?

- Evita *data races* al sobrescribir delta.
- Garantiza un máximo local por hilo, combinado de forma segura al final.

Speedup — parallel for

Malla	Hilos	T (s)	Speedup
512 ²	1	176.55	1.00×
512 ²	16	11.82	14.93×
512 ²	32	6.69	26.38×
512 ²	64	5.48	32.22×
1024 ²	1	1822.81	1.00×
1024 ²	16	117.20	15.55×
1024 ²	32	62.51	29.16×
1024 ²	64	41.49	43.93×

Veredicto Arquitectónico

El *speedup* **sigue creciendo** con la malla: 32,2× en 512² vs **43,9×** en 1024². A mayor problema, mayor fracción paralela efectiva.

Act. 2 — collapse(2): Fusión de Bucles

```

1 #pragma omp parallel for collapse(2) reduction(max:delta)
2 for (int i = 1; i < M; ++i) {
3     for (int j = 1; j < N; ++j) {
4         // ... diferencias finitas ...
5     }
6 }

```

Listing 3: Espacio de iteración fusionado

¿Qué hace collapse(2)?

Fusiona bucles en un espacio lineal único de $(M-1) \times (N-1)$ iteraciones, aumentando la granularidad para el *scheduler*.

Comparativa vs parallel for (64 hilos)

Malla	Variante	T (s)	Speedup
512 ²	par. for	5.48	32.22×
512 ²	collapse(2)	5.78	30.54×
1024 ²	par. for	41.49	43.93×
1024 ²	collapse(2)	44.62	40.85×

¿Mejóro? No.

Introduce *overhead* por el recálculo de índices.
Consistentemente más lento en todas las mallas donde $M \gg p$.

¿Cuándo sí usarlo?

Solo cuando el bucle externo tiene muy pocas iteraciones ($M \approx p$).

Act. 3 — sections: Inicialización en Paralelo

```

1 // Memoria reservada ANTES de la región paralela
2 std::vector<std::vector<double>> T(M+1, std::vector<double>(N+1, 0.0))
3 ;
4 std::vector<std::vector<double>> source(M+1, std::vector<double>(N+1,
5     0.0));
6
7 #pragma omp parallel sections
8 {
9     #pragma omp section
10     initialize_boundaries(M, N, T, h, k);
11
12     #pragma omp section
13     poisson_source(M, N, source);
14 }

```

Listing 4: Bifurcación de inicialización

Speedup — sections (64 hilos)

Malla	T (s)	Speedup
512 ²	6.006	29.40×
1024 ²	47.962	38.00×

¿Tuviste que reordenar? **Sí**

Para evitar *Seg-Faults*, la memoria debe asignarse estáticamente en el hilo maestro.

¿Vale la pena? **No**

- Inicialización $\mathcal{O}(N^2)$ es trivial.
- El *overhead* de fork-join excede el beneficio.
- Ideal solo para I/O asimétrico.

Act. 4 — Control Explícito y schedule

```

1 #pragma omp parallel // Fork explícito
2 {
3     #pragma omp for schedule(static) reduction(max:delta)
4     for (int i = 1; i < M; ++i) {
5         for (int j = 1; j < N; ++j) {
6             // Discretización de Poisson
7         }
8     }
9 } // Join implícito

```

Listing 5: Región paralela separada del bucle

static vs dynamic

- **static:** Reparto en compilación. Ideal para FLOPs uniformes (como Poisson).
- **dynamic:** Reparto en ejecución. Ideal para cargas desbalanceadas.

Speedup — schedule(static) (64 hilos)

Malla	T (s)	Speedup
512 ²	5.517	32.00×
1024 ²	40.684	44.80×

Por qué NO se usó dynamic

Cada celda ejecuta la misma carga. Forzar asignación dinámica inunda el bus con *overhead* de control.

Ventaja parallel + for

No recrear el equipo de hilos en cada iteración del while reduce latencia acumulada. En 1024² logra el **mejor speedup** de todas las directivas: **44,8×**.

Act. 5 — nowait, barrier, single, critical

```

1 double local_delta = 0.0;
2
3 #pragma omp for nowait
4 for (int i = 1; i < M; ++i)
5     for (int j = 1; j < N; ++j) {
6         /* calc diff */
7         if (diff > local_delta) local_delta = diff;
8     }
9
10 #pragma omp critical
11 { if (local_delta > delta) delta = local_delta; }
12
13 #pragma omp barrier
14
15 #pragma omp single
16 { iterations++; }

```

Listing 6: Sincronización manual

Speedup — Sincronización manual (64 hilos)

Malla	T (s)	Speedup
512 ²	7.130	24.76×
1024 ²	42.519	42.87×

nowait: ¿hubo cambio?

El beneficio fue anulado por el embotellamiento del critical y la barrier posterior. Peor que reduction en todas las mallas.

Roles Clave

- **single:** Evita que iterations sume p veces por barrido.
- **critical:** Solo para estructuras complejas no soportadas por hardware.

Act. 6 — critical vs atomic: Software vs Hardware

```

1 // Estrategia 1: Software
2 #pragma omp critical(update_delta)
3 {
4     if (local_delta > delta) delta = local_delta;
5 }
6
7 // Estrategia 2: Hardware
8 #pragma omp atomic
9 iterations++;

```

Listing 7: Sincronización escalares

Comparativa (64 hilos)

Malla	crit. (s)	atom. (s)	Mejora
64 ²	0.158	0.092	+41.5 %
128 ²	0.515	0.274	+46.9 %
256 ²	1.895	1.129	+40.4 %
512 ²	8.579	6.608	+23.0 %
1024 ²	51.138	49.287	+3.6 %

Mecanismo Interno

- **critical:** Mutexes del S.O. Genera *context switch* (costoso).
- **atomic:** Instrucciones nativas en chip (Ej. LOCK INC). Ejecuta en **~3-5 ciclos**.

Tendencia con la escala

La ventaja de **atomic** se reduce a mayor malla: en 1024² el cómputo domina sobre la sincronización. Aun así, **atomic** es siempre preferible.

Regla General

atomic para contadores/sumas simples. **critical** solo para bloques multilínea o memoria dispersa.

Act. 7 — Paralelismo de Tareas (task)

```

1  const int BS = 16; // Block Size
2  #pragma omp parallel shared(T,source,delta)
3  {
4      #pragma omp single
5      {
6          for (int bi=1; bi<M; bi+=BS)
7              for (int bj=1; bj<N; bj+=BS)
8                  #pragma omp task firstprivate(bi,bj) shared(T,delta)
9                  {
10                     double loc_d = 0.0;
11                     for (int i=bi; i<min(bi+BS,M); i++)
12                         for (int j=bj; j<min(bj+BS,N); j++)
13                             { /* calc diff */ }
14
15                     #pragma omp critical
16                     if (loc_d > delta) delta = loc_d;
17                 }
18             } // taskwait implícito
19             iterations++;
20 }

```

Listing 8: División por bloques

Colapso de Rendimiento (64 hilos)

Malla	T (s)	Speedup
64^2	0.427	0.21×
256^2	60.200	0.25×
512^2	380.458	0.46×
1024^2	8064.8	0.23×

par. for en 1024^2 : 41.49 s (43.9×

¿Por qué colapsa?

En 1024^2 se generan **4 096** tareas por iteración. El *scheduler* consume todo el tiempo encolando y despachando, no calculando.

¿Cuándo sí usar task?

Estructuras irregulares: grafos, árboles, recursividad.
Matrices densas → parallel for.

Tabla Comparativa — Malla 1024×1024 , 64 Hilos

Versión	Directiva	Tiempo	Speedup	Diagnóstico HPC
Secuencial	N/A	1822.81 s	1.00×	Baseline de normalización.
Schedule	<code>schedule(static)</code>	40.68 s	44.80×	Superior. Partición estática ideal para dominio regular.
Parallel For	<code>parallel for</code>	41.49 s	43.93×	Mínima intervención. Balance óptimo.
Sync	<code>nowait+barrier</code>	42.52 s	42.87×	Overhead de critical anula beneficio de nowait.
Sections	<code>sections</code>	47.96 s	38.00×	Fork-join no justificado para $O(N^2)$.
Collapse	<code>collapse(2)</code>	44.62 s	40.85×	Leve penalización por índices lineales.
Atomic	<code>atomic</code>	49.29 s	36.98×	Ventaja sobre critical se reduce en mallas grandes.
Critical	<code>critical</code>	51.14 s	35.64×	Penalización por <i>lock contention</i> .
Tareas	<code>task</code>	8064.8 s	0.23×	Colapso Crítico. 4096 tareas/iter asfixian el scheduler.

Regla de Oro

Para EDPs en dominios regulares:

`schedule(static)` / `parallel for` + `reduction` >>
todo lo demás.

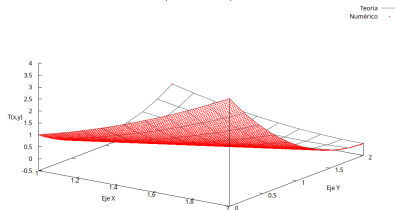
Hallazgo en 1024^2

A diferencia de mallas pequeñas, el *speedup* sigue creciendo con 64 hilos. El cuello de botella se desplaza de la comunicación al cómputo puro.

Validación Numérica de la Solución

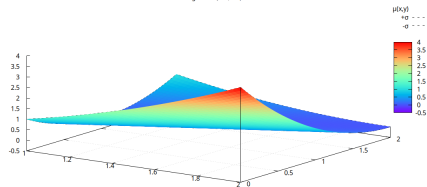
Validación de Rigor: data/solucion_poisson_tomic.dat

Superficie: Teoría vs Cómputo

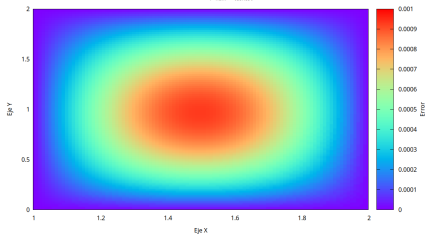


Superficie Promedio y Banda de Incertidumbre σ

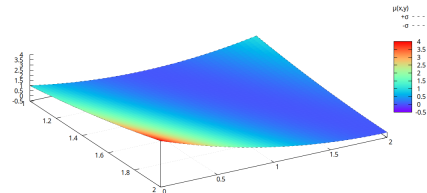
Vista general (60°, 35°)



Error Absoluto Local: $|T_{Num} - T_{Teorico}|$



Vista lateral (25°, 60°)



Conclusiones Generales

1. Paradigma Dominante

- `schedule(static)` y `parallel for + reduction` son superiores.
- La uniformidad geométrica permite distribución estática ideal.
- **Speedup máx:** $44,8\times$ (64 hilos, malla 1024^2).

2. Escalabilidad Creciente

- A mayor malla, mayor fracción paralela α .
- 512^2 : $32,2\times \rightarrow 1024^2$: $43,9\times$.
- El overhead relativo disminuye al crecer el problema.

3. Falla del Modelo de Tareas

- En 1024^2 : 4 096 tareas/iteración $\rightarrow$ colapso total.
- Rendimiento $194\times$ peor que `parallel for`.
- Usar solo para grafos o recursividad.

4. Validez Numérica

- La paralelización **no afecta la física**.
- 100 % de consistencia en el esquema de diferencias finitas iterativo.

Veredicto final: `schedule(static)` y `parallel for` son la solución óptima para EDPs.

Camilo Huertas · Sebastián Rodríguez

Programa Académico de Física
Universidad Distrital Francisco José de Caldas

Docente: Carlos Andrés Gómez Vasco | Sistemas Distribuidos